Documentation

Language: Swift

Framework

Combine

Customize handling of asynchronous events by combining event-processing operators.

iOS 13.0+ iPadOS 13.0+ Mac Catalyst 13.0+ macOS 10.15+ tvOS 13.0+ visionOS 1.0+ watchOS 6.0+

Overview

The Combine framework provides a declarative Swift API for processing values over time. These values can represent many kinds of asynchronous events. Combine declares *publishers* to expose values that can change over time, and *subscribers* to receive those values from the publishers.

- The `Publisher` protocol declares a type that can deliver a sequence of values over time. Publishers have *operators* to act on the values received from upstream publishers and republish them.
- At the end of a chain of publishers, a `Subscriber` acts on elements as it receives them. Publishers only emit values when explicitly requested to do so by subscribers. This puts your subscriber code in control of how fast it receives events from the publishers it's connected to.

Several Foundation types expose their functionality through publishers, including `Timer`, `NotificationCenter`, and `URLSession`. Combine also provides a built-in publisher for any property that's compliant with Key-Value Observing.

You can combine the output of multiple publishers and coordinate their interaction. For example, you can subscribe to updates from a text field's publisher, and use the text to perform URL requests. You can then use another publisher to process the responses and

use them to update your app.

By adopting Combine, you'll make your code easier to read and maintain, by centralizing your event-processing code and eliminating troublesome techniques like nested closures and convention-based callbacks.

Topics

Essentials

Receiving and Handling Events with Combine

Customize and receive events from asynchronous sources.

Publishers

`protocol` **Publisher**

Declares that a type can transmit a sequence of values over time.

`enum` **Publishers**

A namespace for types that serve as publishers.

`struct` **AnyPublisher**

A publisher that performs type erasure by wrapping another publisher.

`struct` **Published**

A type that publishes a property marked with an attribute.

`protocol` **Cancellable**

A protocol indicating that an activity or action supports cancellation.

`class` **AnyCancellable**

A type-erasing cancellable object that executes a provided closure when canceled.

Convenience Publishers

`class` **Future**

A publisher that eventually produces a single value and then finishes or fails.

`struct` **Just**

A publisher that emits an output to each subscriber just once, and then finishes.

`struct` **Deferred**

A publisher that awaits subscription before running the supplied closure to create a publisher for the new subscriber.

`struct` **Empty**

A publisher that never publishes any values, and optionally finishes immediately.

`struct` **Fail**

A publisher that immediately terminates with the specified error.

`struct` **Record**

A publisher that allows for recording a series of inputs and a completion, for later playback to each subscriber.

Connectable Publishers

Controlling Publishing with Connectable Publishers

Coordinate when publishers start sending elements to subscribers.

`protocol` **ConnectablePublisher**

A publisher that provides an explicit means of connecting and canceling publication.

Subscribers

Processing Published Elements with Subscribers

Apply back pressure to precisely control when publishers produce elements.

`protocol` Subscriber

A protocol that declares a type that can receive input from a publisher.

`enum` Subscribers

A namespace for types that serve as subscribers.

`struct` AnySubscriber

A type-erasing subscriber.

`protocol` Subscription

A protocol representing the connection of a subscriber to a publisher.

`enum` Subscriptions

A namespace for symbols related to subscriptions.

Subjects

`protocol` Subject

A publisher that exposes a method for outside callers to publish elements.

`class` CurrentValueSubject

A subject that wraps a single value and publishes a new element whenever the value changes.

`class` PassthroughSubject

A subject that broadcasts elements to downstream subscribers.

Schedulers

`protocol` Scheduler

A protocol that defines when and how to execute a closure.

`struct` ImmediateScheduler

A scheduler for performing synchronous actions.

`protocol` SchedulerTimeIntervalConvertible

A protocol that provides a scheduler with an expression for relative time.

Combine Migration

Routing Notifications to Combine Subscribers

Deliver notifications to subscribers by using notification centers' publishers.

Replacing Foundation Timers with Timer Publishers

Publish elements periodically by using a timer.

Performing Key-Value Observing with Combine

Expose KVO changes with a Combine publisher.

Using Combine for Your App's Asynchronous Code

Apply common patterns to migrate your closure-based, event-handling code.

Observable Objects

`protocol` ObservableObject

A type of object with a publisher that emits before the object has changed.

`class` ObservableObjectPublisher

A publisher that publishes changes from observable objects.

Asynchronous Publishers

`struct` AsyncPublisher

A publisher that exposes its elements as an asynchronous sequence.

`struct` AsyncThrowingPublisher

A publisher that exposes its elements as a throwing asynchronous sequence.

Encoders and Decoders

`protocol` TopLevelEncoder

A type that defines methods for encoding.

`protocol` TopLevelDecoder

A type that defines methods for decoding.

Debugging Identifiers

`protocol` CustomCombineIdentifierConvertible

A protocol for uniquely identifying publisher streams.

`struct` CombineIdentifier

A unique identifier for identifying publisher streams.